

Supervised Learning: Regression and Classification

A beginner notebook for Course 1 of the Machine Learning Specialization. Plain words, pictures, and only the maths you need.

Read this line first. Every model in this notebook does the same four things: it makes a guess with a formula, it measures how wrong the guess is with a **cost function**, it nudges its numbers to be less wrong with **gradient descent**, and it repeats until the cost stops falling. Learn that loop once and every chapter becomes a small variation of it.

0. Quick recap: supervised learning

In supervised learning you give the computer examples where the correct answer (the **label**) is already known. It learns the connection between inputs and answers, then predicts answers for new inputs. There are two types:

Type	The answer is	Example	Model in this notebook
Regression	a number on a scale	house price, exam score	Linear regression
Classification	a category from a small list	spam or not, tumor benign or malignant	Logistic regression

Words used everywhere in this notebook

Word or symbol	Plain meaning
Feature (x)	An input column, for example house size.
Target (y)	The answer column, for example house price.
Training set	The table of solved examples the model learns from.
m	Number of training examples (rows).
n	Number of features (input columns).
$(x^{(i)}, y^{(i)})$	Row number i . The (i) is a row index, not a power.
Parameters (w, b)	The numbers the model adjusts while learning. Also called weights (w) and bias (b).
$\hat{y}$ (y-hat)	The model's prediction. y is the real answer, $\hat{y}$ is the guess.

Course 1 map and where each lesson lives in this notebook

Optional labs, scikit-learn labs, the Jupyter lessons and the optional interview video are left out. Every required video lesson is covered.

Week	Required lessons	Your status	Section
1	Linear regression model, cost function, gradient descent, learning rate	Done	1.1 to 1.3
2	Multiple features, vectorization, gradient descent for multiple regression	Done	1.4
2	Feature scaling, checking convergence, choosing the learning rate	Done	1.5
2	Feature engineering, polynomial regression	Done	1.6
3	Motivations, logistic regression, decision boundary, logistic cost (full and simplified), gradient descent	Upcoming	2.1 to 2.5
3	Overfitting, addressing it, regularized cost, regularized linear and logistic regression	Upcoming	3.1 to 3.4

1. REGRESSION

1.1 Linear regression with one variable

Why we need it: Many real questions are 'how much?' questions. Linear regression is the simplest model that learns the answer from past examples instead of hand-written rules, and every later model in this course builds on it.

Linear regression predicts a number by drawing the **best straight line** through the data. With one feature the model is:

$$f_{w,b}(x) = wx + b$$

f(x)	The prediction ($\hat{y}$) for input x .
w	Slope: how much the prediction rises when x grows by 1.
b	Intercept: the prediction when x is 0 (where the line crosses the y axis).

Dataset A - House prices. One feature (size) and one target (price).

Row i	Size in 1000 sq ft (x)	Price in \$1000s (y)
1	0.8	200
2	1.0	245
3	1.2	260
4	1.5	330
5	1.8	370
6	2.0	400
7	2.3	470

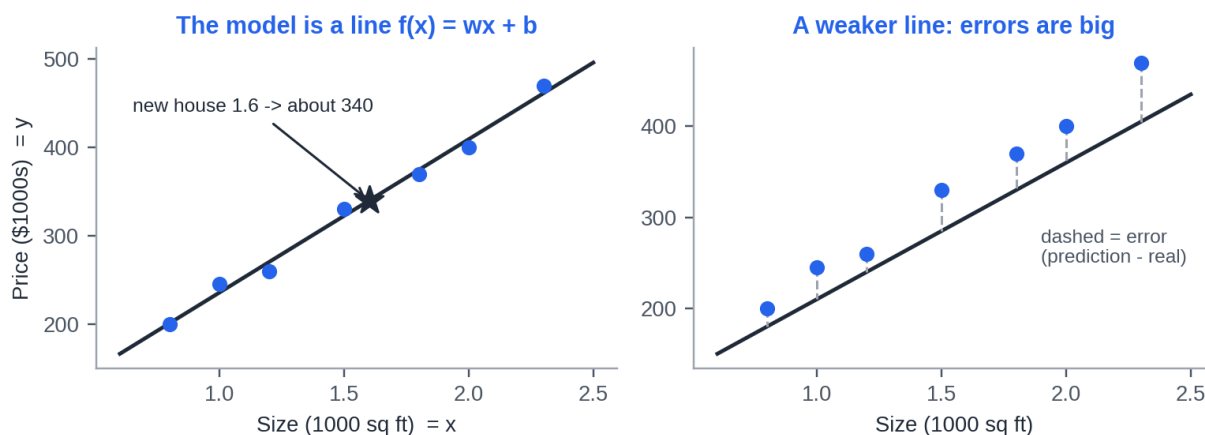


Figure 1.1: Left, the best line and a prediction for a new house. Right, a worse line with its errors drawn as dashed gaps.

Learning means finding the w and b that make the line sit as close to all points as possible. To do that we need a way to score a line. That score is the cost function.

1.2 Cost function

Why we need it: Without a score we cannot tell a good line from a bad one. The cost function turns 'how wrong is this line?' into one number that the computer can try to make smaller.

The cost function measures **how wrong the line is on the whole training set**. For linear regression it is the squared error cost:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$f(x^{(i)}) - y^{(i)}$	The error on row i : prediction minus real answer.
$()^2$	Squaring makes every error positive and punishes big errors much more than small ones.
Σ and $1/m$	Add the squared errors of all m rows, then average them.
the extra $1/2$	Only makes the calculus neater later. It does not change which line is best.

Small cost = good line. Large cost = bad line. The goal of training is simply: find w and b that make J as small as possible.

Worked example

Take three points (1,1), (2,2), (3,3) and fix $b = 0$. Try $w = 0.5$. Predictions are 0.5, 1, 1.5. Errors are -0.5, -1, -1.5. Squares are 0.25, 1, 2.25, which sum to 3.5. Cost = $3.5 / (2 \times 3) = 0.58$. Try $w = 1$ and every prediction is perfect, so the cost is 0.

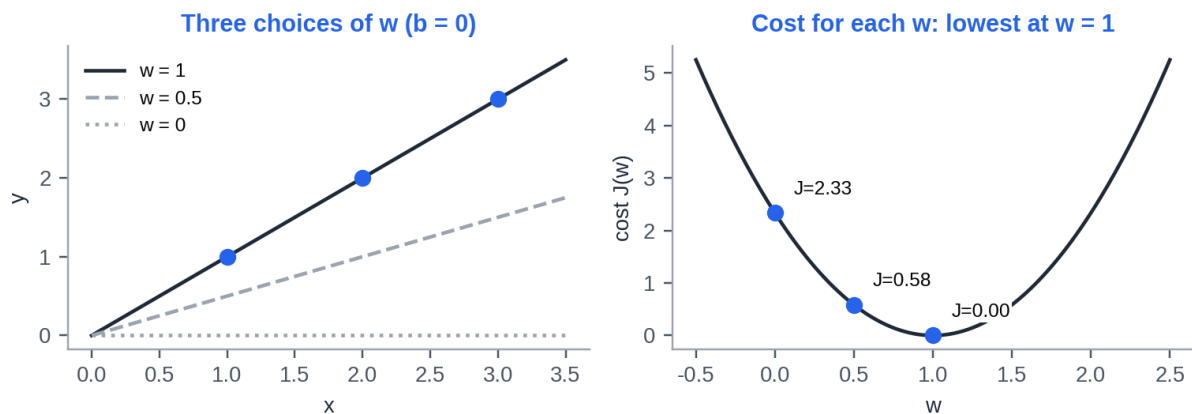


Figure 1.2: Each line on the left is one dot on the cost curve on the right. The bottom of the bowl is the best w .

With two parameters, J becomes a 3D bowl over (w, b) . We usually draw it from above as a **contour plot**: each oval joins points of equal cost, like height lines on a map. The centre of the smallest oval is the best line.

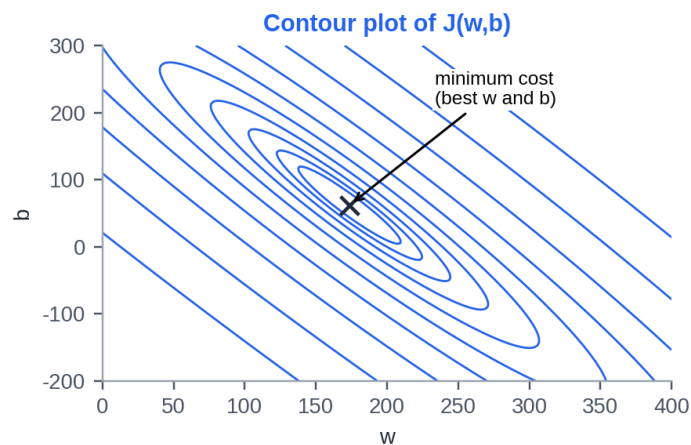


Figure 1.3: Contour plot of the cost for Dataset A. Every point is one possible line.

1.3 Gradient descent and the learning rate

Why we need it: There are endless possible values of w and b , so we cannot try them all by hand. Gradient descent finds good values automatically, one small step at a time.

Picture standing on a hill in fog: you look at the slope under your feet, take a small step downhill, and repeat until the ground is flat.

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) \qquad b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

=	Here it means 'update': replace the old value with the new one.
α (alpha)	The learning rate: the size of each step, a small positive number like 0.01.
$\partial J / \partial w$	The derivative: the slope of the cost in the w direction. It says which way is uphill and how steep it is.

Why the minus sign works. If the slope is positive, subtracting it moves w left (downhill). If the slope is negative, subtracting a negative moves w right (also downhill). At the bottom the slope is 0, so the updates stop by themselves. Steps also get smaller near the bottom because the slope gets flatter, even with a fixed α .

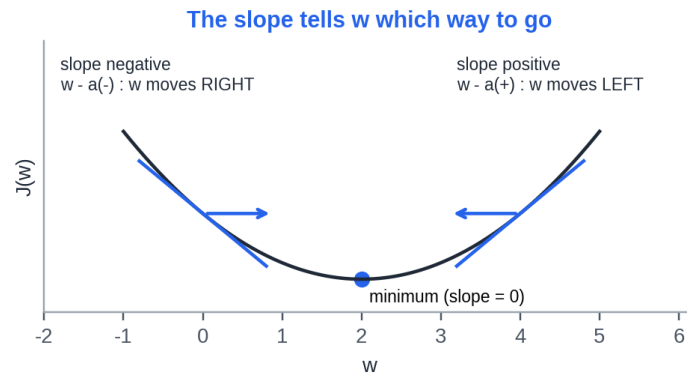


Figure 1.4: The sign of the slope always points the update downhill.

Update both at the same time. Compute the new w and new b using the old values, then assign both together (a simultaneous update). Updating w first and then using the new w to compute b is a common mistake.

Choosing the learning rate α

Why we need it: Gradient descent knows which direction to step, but not how far. The learning rate sets the step size, and a bad choice makes training painfully slow or makes it fail completely.

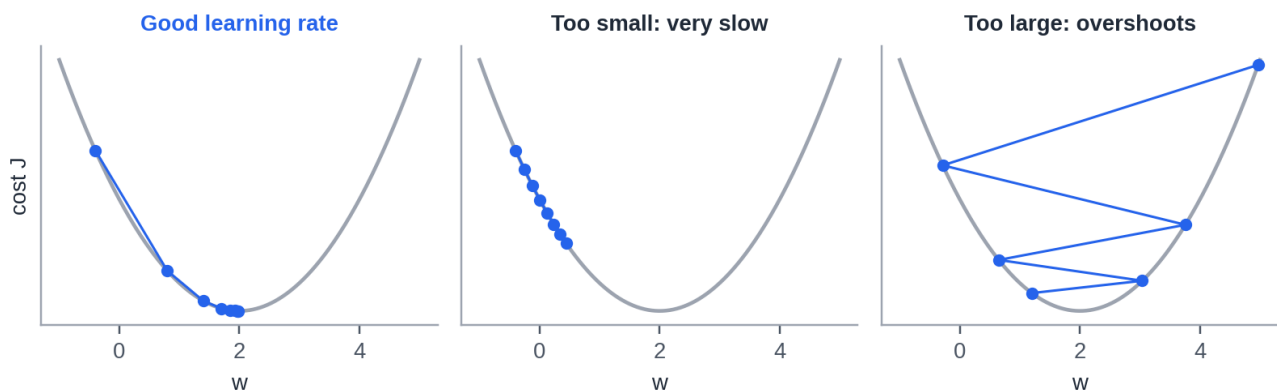


Figure 1.5: Same bowl, same start, three learning rates.

- **Too small:** it works but needs a huge number of steps.
- **Too large:** it jumps over the minimum and can move further away each step, so the cost grows (diverges).
- **Good:** quick, steady progress down to the minimum.

Gradient descent for linear regression

When you work out the two derivatives for the squared error cost you get these update rules. Repeat them until convergence (the cost stops dropping):

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)} \qquad b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

In words: **average error times input** adjusts w , and **average error** adjusts b . Because each step looks at all m rows, this version is called **batch gradient descent**. The squared error cost of linear regression is always one single bowl

(it is **convex**), so there is no false bottom to get stuck in.

One step by hand. Points (1,1), (2,2), (3,3), b fixed at 0, start at $w = 0$, $\alpha = 0.1$. All predictions are 0, so the errors are -1, -2, -3. Slope = $(1/3)(-1 \times 1 - 2 \times 2 - 3 \times 3) = -14/3 = -4.67$. New $w = 0 - 0.1 \times (-4.67) = \mathbf{0.47}$. One step moved w about halfway toward the best value of 1.

Quick self-test A

1. Using the points (1,1), (2,2), (3,3) and $b = 0$, what is the cost J when $w = 2$?
2. The slope $\partial J / \partial w$ at your current w is negative. After one update, does w get bigger or smaller?
3. You are at the exact minimum of J . What does one more gradient descent step do?
4. The cost goes up after every iteration. What is the most likely cause?

1.4 Multiple features and vectorization

Why we need it: One input rarely explains the answer. A house price depends on size, bedrooms, age and more, so the model needs a weight for each input.

Real predictions use more than one input. With n features, each feature gets its own weight:

$$f_{\vec{w}, b}(\vec{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \vec{w} \cdot \vec{x} + b$$

x_j	Feature number j (for example x_2 = bedrooms).
w_j	The weight for feature j : how much the price changes when that feature grows by 1, with the others fixed.
$\mathbf{x}$ with an arrow	A vector: the whole row of features written as one list, $[x_1, x_2, \dots, x_n]$.
$\mathbf{w} \cdot \mathbf{x}$	The dot product: multiply matching pairs and add them up.

House	Size (1000 sq ft) x_1	Bedrooms x_2	Floors x_3	Age in years x_4	Price \$1000s (y)
1	2.10	5	1	45	460
2	1.42	3	2	40	232
3	1.53	3	2	30	315
4	0.85	2	1	36	178
5 (new)	1.20	3	2	10	= ? to predict

Worked prediction. Suppose training gave $w = [150, 10, 20, -2]$ and $b = 50$. For house 5: $150 \times 1.2 + 10 \times 3 + 20 \times 2 + (-2) \times 10 + 50 = 180 + 30 + 40 - 20 + 50 = \mathbf{280}$, so about \$280,000. The negative weight on age means older houses are predicted cheaper.

Vectorization

Vectorization means writing the calculation as one operation on whole lists instead of a loop over each feature. In Python with NumPy the whole prediction is one line: `f = np.dot(w, x) + b`. It is shorter to write, and it runs much faster because the computer processes many numbers in parallel instead of one at a time. With thousands of features and rows this difference is huge.

Gradient descent with multiple features

Nothing new in the idea. Every weight gets the same kind of update, using its own feature:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \quad (j = 1 \dots n) \quad b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

1.5 Making gradient descent work well in practice

Feature scaling

If one feature ranges from 300 to 2000 (size in sq ft) and another from 0 to 5 (bedrooms), a tiny change in the size weight moves the prediction a lot, while the bedroom weight needs large changes. The cost bowl becomes a long, thin valley and gradient descent zig-zags slowly. **Feature scaling** puts all features on similar ranges (roughly -1 to 1) so the bowl is round and the path is direct.

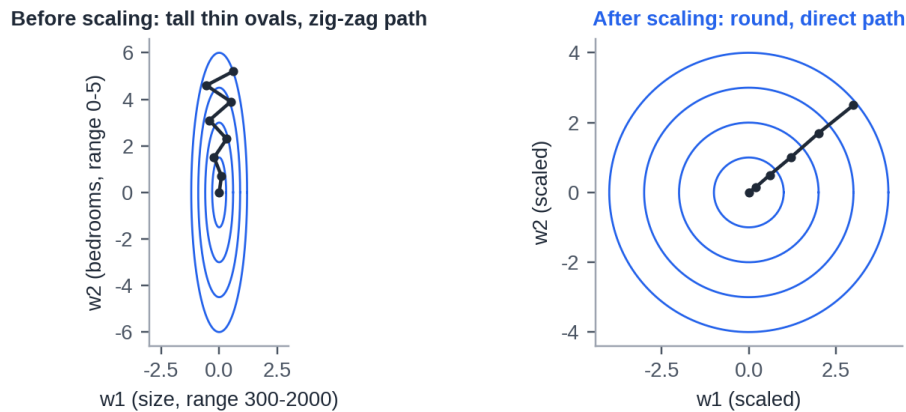


Figure 1.6: Unscaled features make a stretched bowl. Scaled features make a round one.

Method	How	Size $x = 1000$ with range 300 to 2000, mean $\mu = 1100$, std $\sigma = 450$
Divide by max	$x / \max$	$1000 / 2000 = 0.50$
Mean normalization	$(x - \mu) / (\max - \min)$	$(1000 - 1100) / 1700 = -0.06$
Z-score normalization	$(x - \mu) / \sigma$	$(1000 - 1100) / 450 = -0.22$

Rule of thumb: ranges like -1 to 1 or -3 to 3 are fine; 0 to 1000 or -0.001 to 0.001 should be rescaled. When unsure, scale. Remember to apply the **same** μ and σ to new data before predicting.

Checking convergence and choosing α

Why we need it: Training runs silently. You need a way to see whether it is actually working, when it has finished, and whether your α is a good choice.

Plot the cost J after each iteration. This **learning curve** is your main health check.

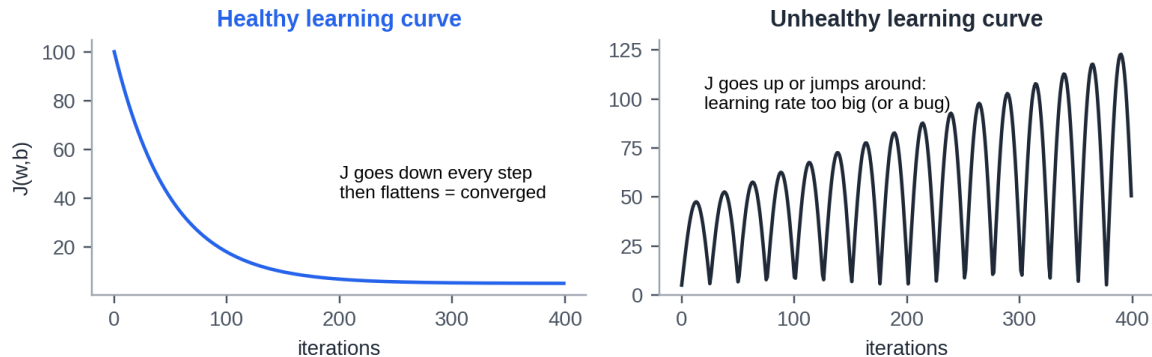


Figure 1.7: Cost should fall on every iteration and then level off.

- **Converged:** the curve is flat. An automatic test: stop when J drops by less than a tiny ϵ (epsilon, such as 0.001) in one iteration.
- **Cost rising or bouncing:** α is too large, or there is a bug in the code. Debug trick: with a very small α , J must go down every iteration. If it still does not, the code is wrong.
- **Picking α :** try values about 3 times apart, for example 0.001, 0.003, 0.01, 0.03, 0.1, and keep the largest one that still decreases J steadily.

1.6 Feature engineering and polynomial regression

Why we need it: A model can only be as good as the features it gets, and real data is often curved. Better features let a simple linear model capture patterns it would otherwise miss.

Feature engineering means creating new, more useful features from the ones you have, using your knowledge of the problem. Example: for a plot of land you have frontage (x_1) and depth (x_2). Price depends more on area, so you add $x_3 = x_1 \times x_2$. The model can now use whichever works best.

Polynomial regression is feature engineering to fit curves. You still use linear regression, but you feed it powers of a feature such as x , x^2 , x^3 or $\sqrt{x}$. The model stays 'linear in w ', yet the line it draws can bend.

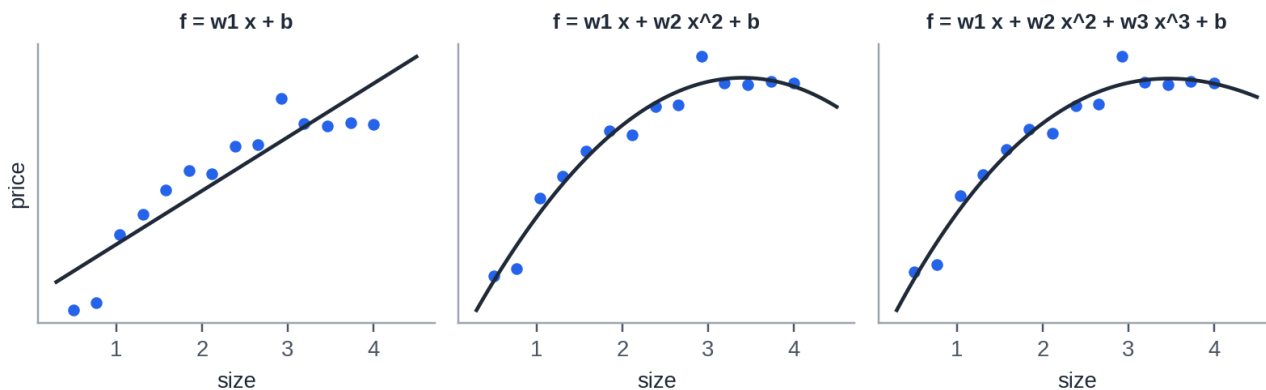


Figure 1.8: Same data. Adding x squared and x cubed lets the model follow the curve.

Feature scaling becomes essential here: if size runs from 1 to 1000, then x^2 runs to 1,000,000 and x^3 to 1,000,000,000. Also, more powers is not automatically better; too many can make the curve wiggle through noise, which is the overfitting problem in Chapter 3.

Where regression is used: house and rent pricing, sales and demand forecasting, predicting how many days a patient will stay in hospital, estimating crop yield from rainfall and fertilizer.

Quick self-test B

1. A model has $w = [2, 3]$, $b = 1$. What does it predict for $x = [4, 5]$?
2. Feature x_1 ranges 0 to 5 and x_2 ranges 100 to 2000. What happens to gradient descent if you do not scale, and which feature is the main problem?
3. You have the length and the width of a room. Suggest one engineered feature for predicting the rent.
4. You add x , x squared and x cubed where x goes from 1 to 1000. Why is feature scaling now very important?

2. CLASSIFICATION

Now the answer is a category. In Course 1 we focus on **binary classification**: only two possible answers, written as **0** (negative class, for example benign, not spam) and **1** (positive class, for example malignant, spam). 'Negative' and 'positive' only mean absent and present, not bad and good.

2.1 Why linear regression fails for classification

You could fit a straight line and say 'predict 1 if the line is above 0.5'. It can look fine at first, but one extra example far to the right tilts the line and moves the cut point, so correct predictions suddenly become wrong. A line also outputs values like -0.4 or 1.7, which make no sense as answers.

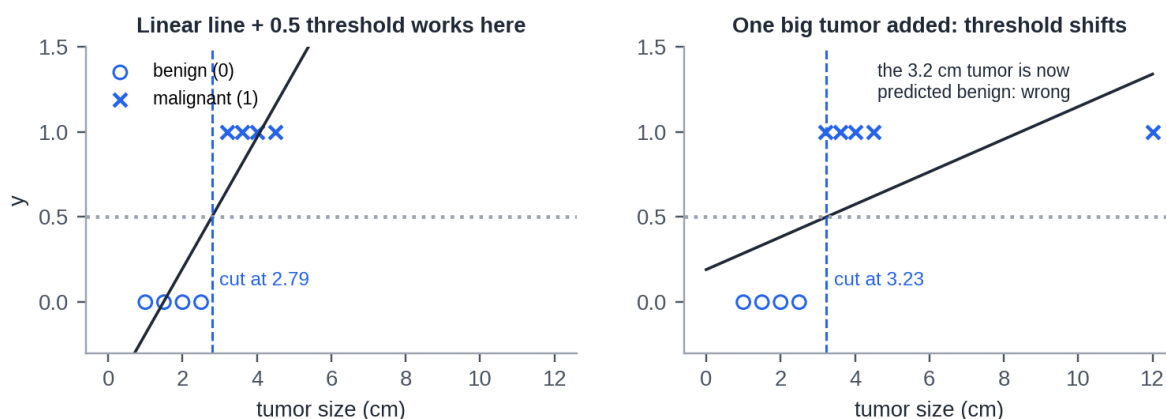


Figure 2.1: Adding one very large (and obviously malignant) tumor moves the cut from 2.79 to 3.23 cm.

2.2 Logistic regression and the sigmoid function

Why we need it: We need a model whose output always stays between 0 and 1, so it can be read as a probability and turned into a clear yes or no.

Logistic regression fixes this by passing the linear formula through the **sigmoid function** g , which squeezes any number into the range 0 to 1:

	$z = \vec{w} \cdot \vec{x} + b$	$g(z) = \frac{1}{1 + e^{-z}}$	$f_{\vec{w}, b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$
z	The same linear score as before. It can be any number.		
e	A math constant, about 2.718.		
g(z)	Large positive z gives nearly 1, large negative z gives nearly 0, z = 0 gives exactly 0.5.		

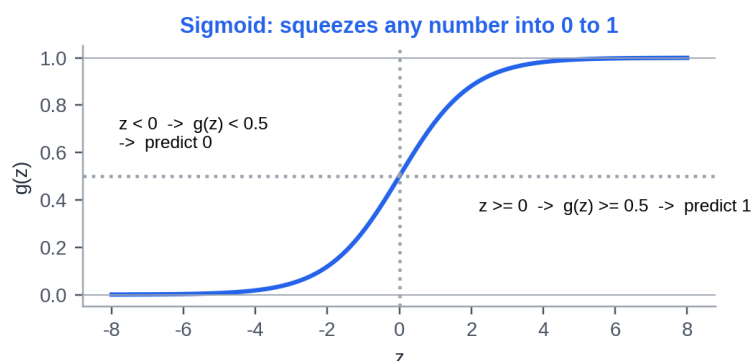


Figure 2.2: The sigmoid curve. Its output reads as a probability.

How to read the output. $f(x)$ is the model's estimated **probability that $y = 1$** . If a tumor gets $f = 0.7$, the model says: 70% chance malignant, 30% chance benign.

2.3 Decision boundary

Why we need it: A probability is not yet a decision. The decision boundary shows exactly where the model switches from predicting 0 to predicting 1, so you can see what it has learned.

To give a final yes or no, we use a threshold of 0.5: predict 1 when $f \geq 0.5$. Since $g(z) \geq 0.5$ exactly when $z \geq 0$, the rule is simply: **predict 1 when $w \cdot x + b \geq 0$** . The line (or curve) where $z = 0$ is the **decision boundary**. It splits the feature space into the 'predict 0' side and the 'predict 1' side.

- **Straight boundary:** with $w_1 = 1$, $w_2 = 1$, $b = -3$ we get $z = x_1 + x_2 - 3$. The boundary is the line $x_1 + x_2 = 3$.
- **Curved boundary:** with polynomial features x_1^2 and x_2^2 , weights 1 and 1, and $b = -1$, the boundary is the circle $x_1^2 + x_2^2 = 1$. More complex features give more complex shapes.

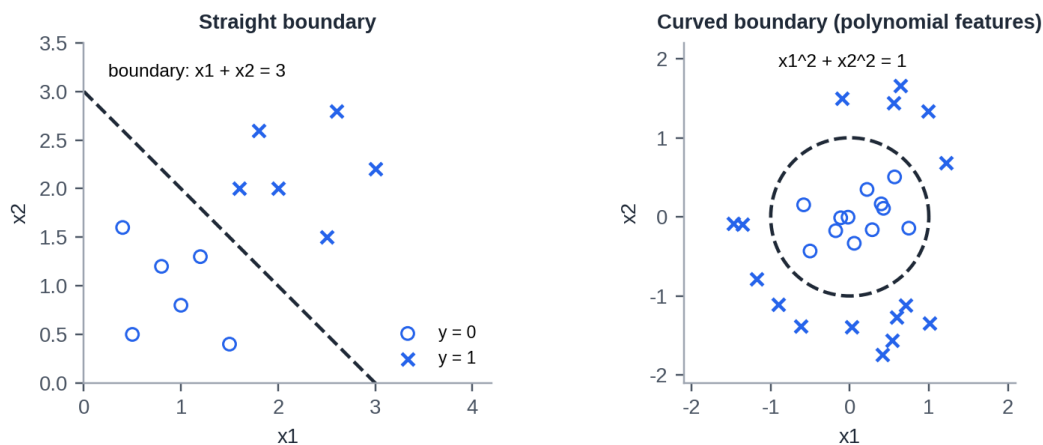


Figure 2.3: Circles are $y = 0$, crosses are $y = 1$. The dashed line is where $z = 0$.

2.4 Cost function for logistic regression

Why not reuse the squared error cost? Because with the sigmoid inside, that cost is no longer one smooth bowl. It becomes a bumpy surface with many **local minima** (false bottoms) where gradient descent can get stuck. So logistic regression uses a different **loss** for each example. (Loss = error on one row. Cost = average loss over all rows.)

$$L = -\log(f(\vec{x})) \quad \text{if } y = 1$$

$$L = -\log(1 - f(\vec{x})) \quad \text{if } y = 0$$

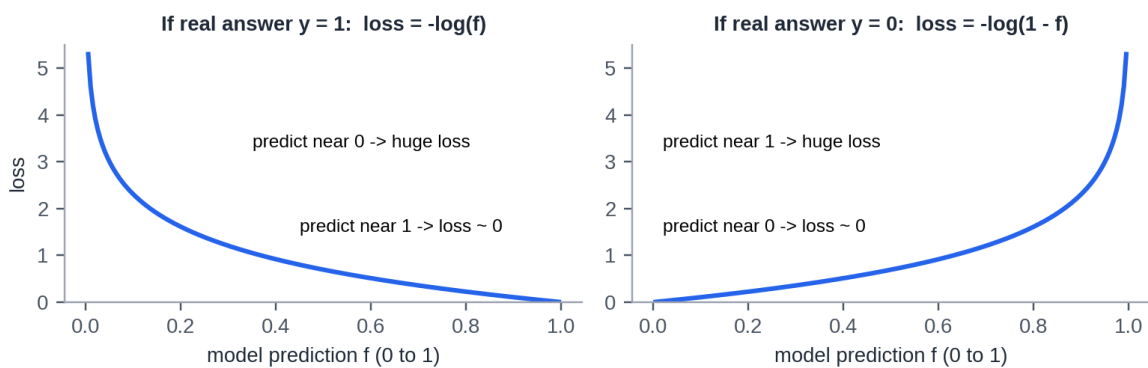


Figure 2.4: Confident and correct costs almost nothing. Confident and wrong costs a lot.

Example. A tumor is malignant ($y = 1$). If the model says $f = 0.9$, loss = $-\log(0.9) = 0.11$ (small). If it says $f = 0.1$, loss = $-\log(0.1) = 2.30$ (large). The loss pushes predictions toward the true label.

Simplified cost function

The two cases can be written as one line. When $y = 1$ the second part becomes 0; when $y = 0$ the first part becomes 0, so it is exactly the same loss:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log f(\vec{x}^{(i)}) + (1 - y^{(i)}) \log (1 - f(\vec{x}^{(i)}))]$$

log The natural logarithm (ln).

the minus sign log of a number between 0 and 1 is negative, so the minus makes the cost positive.

This cost is convex (one bowl), so gradient descent can reliably reach the best w and b .

2.5 Gradient descent for logistic regression

After working out the derivatives, the update rules look **exactly the same** as for linear regression:

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \quad b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

Same look, different model. The only difference is what f means. In linear regression $f = w \cdot x + b$. In logistic regression $f = g(w \cdot x + b)$, the sigmoid of it. Everything from Chapter 1 still applies: simultaneous updates, learning curves to check convergence, choosing α , feature scaling and vectorization.

Where classification is used: spam filters, fraud detection on card payments, screening tests (disease present or absent), predicting whether a customer will cancel, and approving or rejecting a loan.

Quick self-test C

1. What is $g(0)$? What does $g(z)$ approach when z is very large and positive?
2. A model has $z = x_1 + x_2 - 3$. What does it predict for the point $(2, 2)$? And for $(1, 1)$?
3. The real label is $y = 1$ and the model outputs $f = 0.01$. Is the loss small or large? Roughly what is it?
4. Why does logistic regression not use the squared error cost?

3. OVERFITTING AND REGULARIZATION

3.1 The problem of overfitting

Why we need it: A model that scores perfectly on its training data can still fail on real data. Spotting underfitting and overfitting is how you know whether to trust a model at all.

A model's real job is **generalization**: doing well on new data it has never seen. There are two ways to fail:

- **Underfitting (high bias):** the model is too simple to capture the pattern, so it does poorly even on the training data. It is 'biased' toward a shape (like a straight line) that does not match reality.
- **Overfitting (high variance):** the model is so flexible that it memorizes the training data, including its noise. Training cost is nearly zero, but predictions on new data are poor, and a slightly different training set would produce a very different model.
- **Just right:** it captures the true pattern and ignores the noise, so it generalizes well.

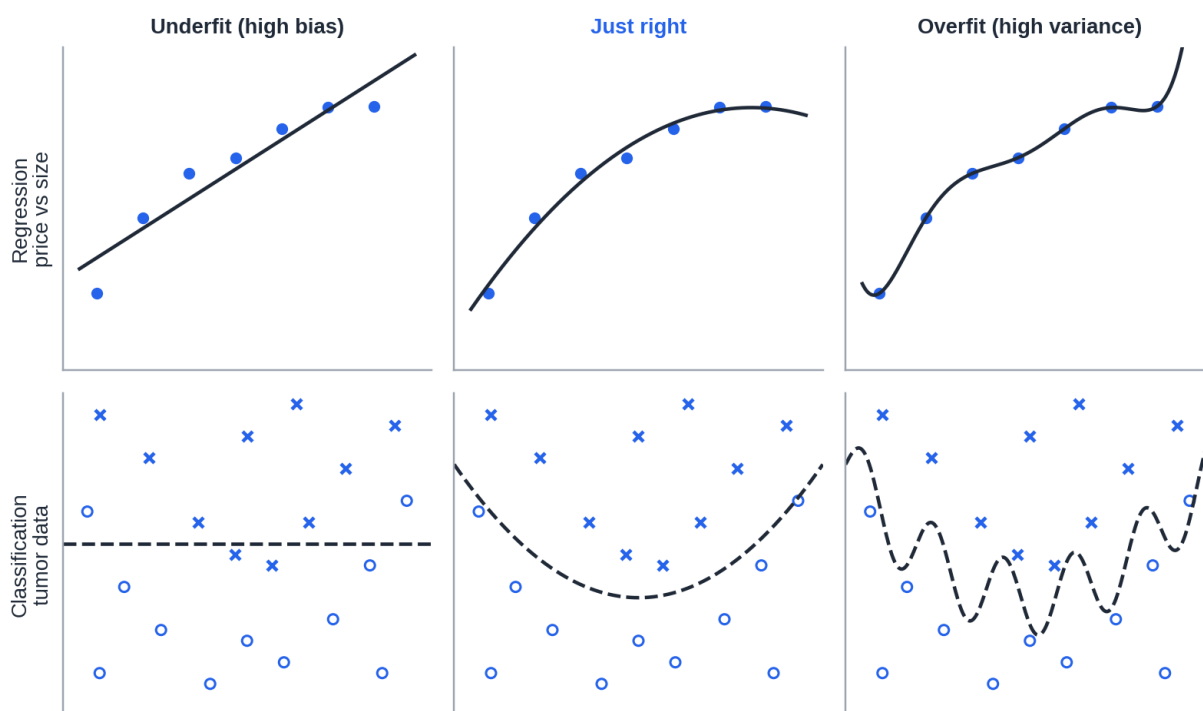


Figure 3.1: The same three situations in regression (top) and classification (bottom).

Think of students before an exam: one did not study enough (underfit), one memorized last year's answers word for word and fails on new questions (overfit), and one understood the ideas (just right).

3.2 Addressing overfitting

Option	What you do	Trade-off
1. Collect more data	Add more training examples	The best fix when possible, but data is not always available
2. Use fewer features	Keep only the most relevant features (feature selection)	You may throw away useful information
3. Regularization	Keep all features but shrink the weights w_j toward small values	Needs a new setting λ to choose; the method the course focuses on

Why small weights help: a large weight on x^4 or x^5 is what creates the wild wiggles. Shrinking those weights flattens the wiggles while still letting every feature contribute a little.

3.3 Cost function with regularization

Why we need it: We want to keep useful features without letting any of them dominate. Regularization gives the model a reason to keep weights small, which smooths out wild curves automatically.

We add a penalty to the cost that grows when the weights grow. Now the model must balance two goals: fit the data, and keep the weights small.

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

first part	The usual squared error cost: fit the data.
second part	The penalty: keep the weights small.
λ (lambda)	The regularization parameter. It sets how strong the penalty is.
$\sum w_j^2$	Sum of all squared weights, from $j = 1$ to n .
b	Usually not penalized. It makes very little difference in practice.

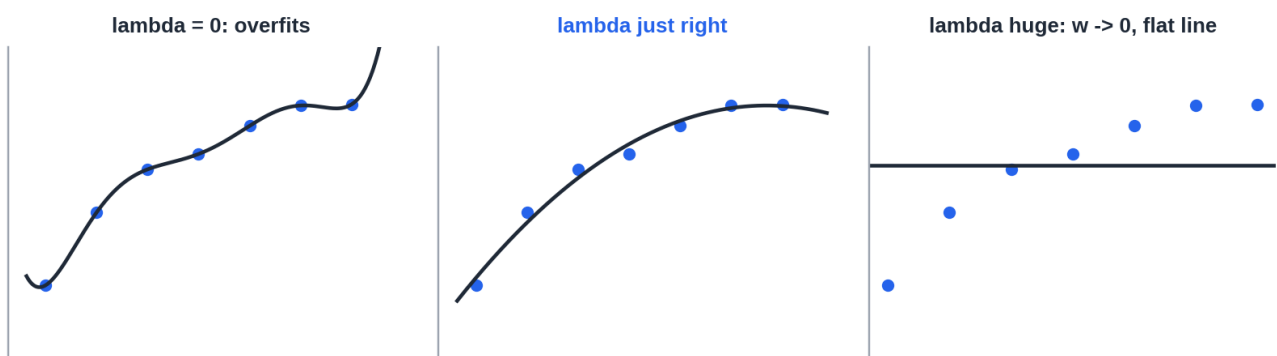


Figure 3.2: λ is a dial between overfitting and underfitting.

- **$\lambda = 0$:** no penalty, same as before, so the model can overfit.
- **λ very large:** all weights are forced to nearly 0, so $f(x) \approx b$, a flat line that underfits.
- **λ in between:** a smooth curve that fits well. Choosing it well is covered later in the specialization.

3.4 Regularized linear and logistic regression

Only the derivative for w_j changes. It gets one extra term, $(\lambda/m)w_j$. The update for b stays exactly the same.

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

What this really does. Rearranged, the update is $w_j = w_j(1 - \alpha\lambda/m) -$ (the usual term). With $\alpha = 0.01$, $\lambda = 1$, $m = 50$, the factor is 0.9998. So on every step each weight is first shrunk by a tiny amount, then updated as usual. This constant gentle shrinking is why regularization is also called **weight decay**.

Regularized logistic regression works the same way: add the same penalty $(\lambda/2m)\sum w_j^2$ to the logistic cost from Section 2.4, and use the same update rule above, remembering that there f is the sigmoid. This lets you use many polynomial features for a curved decision boundary without the boundary becoming wildly twisted.

Quick self-test D

1. A model has almost zero cost on the training set but makes poor predictions on new data. What is the problem called?
2. What happens to the model if you set λ to a huge value like 10,000,000,000?
3. List the three main ways to address overfitting.
4. In regularized gradient descent, which parameter is updated exactly as before?

4. ONE-PAGE SUMMARY

Idea	Formula	Remember
Linear model	$f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$	Predicts a number. One feature: $f = wx + b$.
Squared error cost	$J = \frac{1}{2m} \sum (f(\vec{x}^{(i)}) - y^{(i)})^2$	Average squared error. Lower is better. Convex.
Gradient descent	$w_j = w_j - \alpha \frac{\partial J}{\partial w_j}, b = b - \alpha \frac{\partial J}{\partial b}$	Step downhill. Update all parameters together.
Gradients	$\frac{1}{m} \sum (f - y)x_j$ and $\frac{1}{m} \sum (f - y)$	Same form for linear and logistic regression.
Learning rate α	α	Too small: slow. Too large: diverges. Try 0.001, 0.003, 0.01...
Z-score scaling	$x_{new} = \frac{x - \mu}{\sigma}$	Put features on similar ranges. Use the same μ, σ on new data.
Polynomial features	$x, x^2, x^3, \sqrt{x}$	Lets a linear model draw curves. Scale them.
Sigmoid	$g(z) = \frac{1}{1 + e^{-z}}$	Output 0 to 1 = probability that $y = 1$.
Logistic model	$f = g(\vec{w} \cdot \vec{x} + b)$	Predict 1 when $f \geq 0.5$, meaning $z \geq 0$.
Decision boundary	$\vec{w} \cdot \vec{x} + b = 0$	Line or curve separating the two predictions.
Logistic loss	$-y \log f - (1 - y) \log(1 - f)$	Confident and wrong is punished heavily. Convex cost.
Regularized cost	$J + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$	Penalize big weights. Do not penalize b .
Regularized update	$w_j = w_j - \alpha [\frac{1}{m} \sum (f - y)x_j + \frac{\lambda}{m} w_j]$	Shrinks each w a little every step.

Diagnose your model in one glance

What you see	What it means	What to do
Cost J rises or jumps during training	α too large, or a bug	Lower α ; test with a tiny α
Cost falls very slowly	α too small or features unscaled	Raise α ; scale features
High error on training data too	Underfitting (high bias)	More or better features, polynomial terms, smaller λ
Low training error, high error on new data	Overfitting (high variance)	More data, fewer features, larger λ

Remember only this if you forget everything else.

Model makes a guess -> cost measures the error -> gradient descent reduces the error -> repeat.
 Number answer: linear regression with squared error. Yes or no answer: logistic regression with log loss.
 Too simple underfits, too flexible overfits, and regularization is the dial in between.

5. ANSWERS TO THE SELF-TESTS

Q	Answer
A1	Predictions 2, 4, 6. Errors 1, 2, 3. Squares sum to 14. $J = 14 / (2 \times 3) = 2.33$.
A2	Bigger. $w - \alpha x$ (negative number) adds a positive amount.
A3	Nothing. The slope is 0, so the update subtracts 0 and w stays the same.
A4	The learning rate α is too large (or there is a bug in the code).
B1	$2 \times 4 + 3 \times 5 + 1 = 8 + 15 + 1 = 24$.
B2	The cost bowl becomes a long, thin valley and gradient descent zig-zags and converges slowly. x_2 (range up to 2000) is the main problem; scale both so they share a similar range.
B3	Area = length $\times$ width. Rent usually depends on floor area more than on either side alone.
B4	x squared reaches 1,000,000 and x cubed reaches 1,000,000,000, so without scaling the ranges are wildly different.
C1	$g(0) = 0.5$. For very large positive z , $g(z)$ gets closer and closer to 1.
C2	(2, 2): $z = 2 + 2 - 3 = 1$, which is ≥ 0 , so predict 1. (1, 1): $z = -1$, which is below 0, so predict 0.
C3	Large. Loss = $-\log(0.01) \approx 4.6$, compared with about 0.1 for a good prediction.
C4	With the sigmoid inside, squared error gives a non-convex cost with many local minima, so gradient descent may get stuck. Log loss is convex.
D1	Overfitting (high variance).
D2	All w_j are pushed to almost 0, so $f(x) \approx b$, a flat line. The model underfits.
D3	Collect more data, use fewer features, and apply regularization.
D4	b. Its update rule has no λ term.